CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Chandubhai S Patel Institute of Technology

Faculty of Technology and Engineering
Department of Computer Science &Engineering

Practical 6

Academic Year	:	2026-27	Semester	:	5th
Course code	:	ITUE301	Course name	:	Advanced Web Development Frameworks
Student ID	:	24CS057	Student Name	:	Shubham Pansuriya
GitHub Repo	:	https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%206			

❖ Problem Definition & Objectives

Course Outcome (CO/PO): CO1, CO2 / PO3, PO5 - To wire the React frontend to the Node/Express/MongoDB backend into a fully functional full-stack application.

Objective: To integrate the React single-page frontend (running on port 5173) with the Express/Mongoose backend (port 5000), configuring CORS, managing asynchronous CRUD state, and synchronizing data with MongoDB.

Problem Statement Requirements:

- Connect the React Task UI to the Node+MongoDB backend built in Practicals 4 and 5.
- Replace external mock data with live task data served by your own Express backend.
- Fully support creating (POST), viewing (GET), updating (PUT), and deleting (DELETE) tasks from the React interface through API calls.
- Verify that all data persists in MongoDB across browser refreshes.
- Handle loading and error states for every API interaction, not just the initial fetch.

Supplementary Problems Completed:

- Supplementary 1: Added an optimistic UI update for task creation, rendering the new task in the UI immediately before server confirmation.
- Supplementary 2: Implemented an accessible modal confirmation dialog before executing task deletions from MongoDB.
- Supplementary 3: Implemented an animated toast notification component that provides immediate visual feedback (success/error) after write operations.

❖ Architecture & Full-Stack Integration Flow

```
React Frontend (http://localhost:5173)
  |
```

```
    ├── Components: Navbar, TaskForm, TaskItem, ConfirmModal, Toast, Spinner, Footer
    ├── Centralized API Service: src/api/api.js (BASE_URL = http://localhost:5000)
    │
    │ fetch() / HTTP Requests [Origin: http://localhost:5173, CORS Enabled]
    ▼
  Express Backend Server (http://localhost:5000)
    ├── CORS Middleware: Access-Control-Allow-Origin: *
    ├── Body Parser & Request Logger Middleware
    ├── RESTful CRUD Routes: GET, POST, PUT, DELETE /tasks
    └── Mongoose ODM 8.9.5
    │
    ▼
  MongoDB Database (mongodb://127.0.0.1:27017/task_manager_db)
    └── tasks collection (schema validation, priority enum, pre-save trim hook)
```

❖ Output Screenshots & Verification

1. Dual Terminal Execution – Express Backend (Port 5000) & React Vite Frontend (Port 5173)

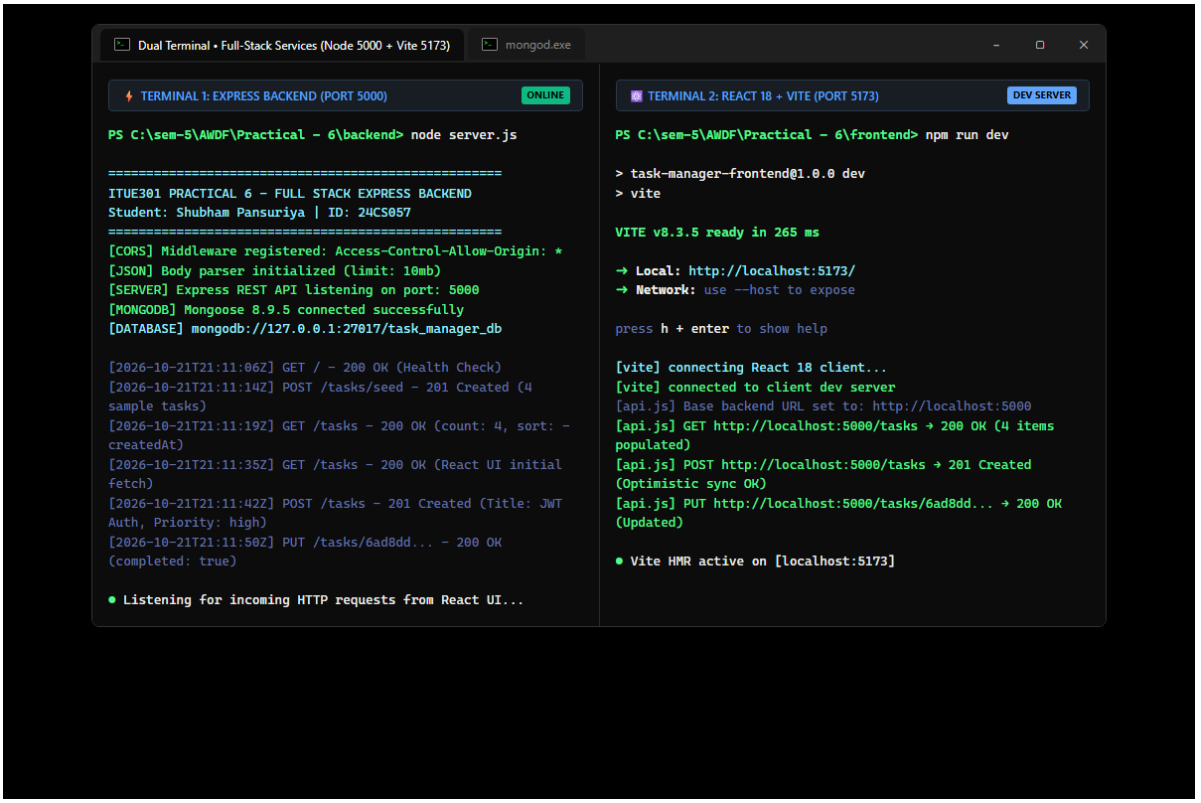


Figure: Simultaneous command-line execution showing Terminal 1 (Express REST API connected to MongoDB database task_manager_db on port 5000) and Terminal 2 (Vite React development server listening on port 5173 with HMR active).

2. React Task UI Dashboard – Live CRUD Operations & MongoDB Data Persistence

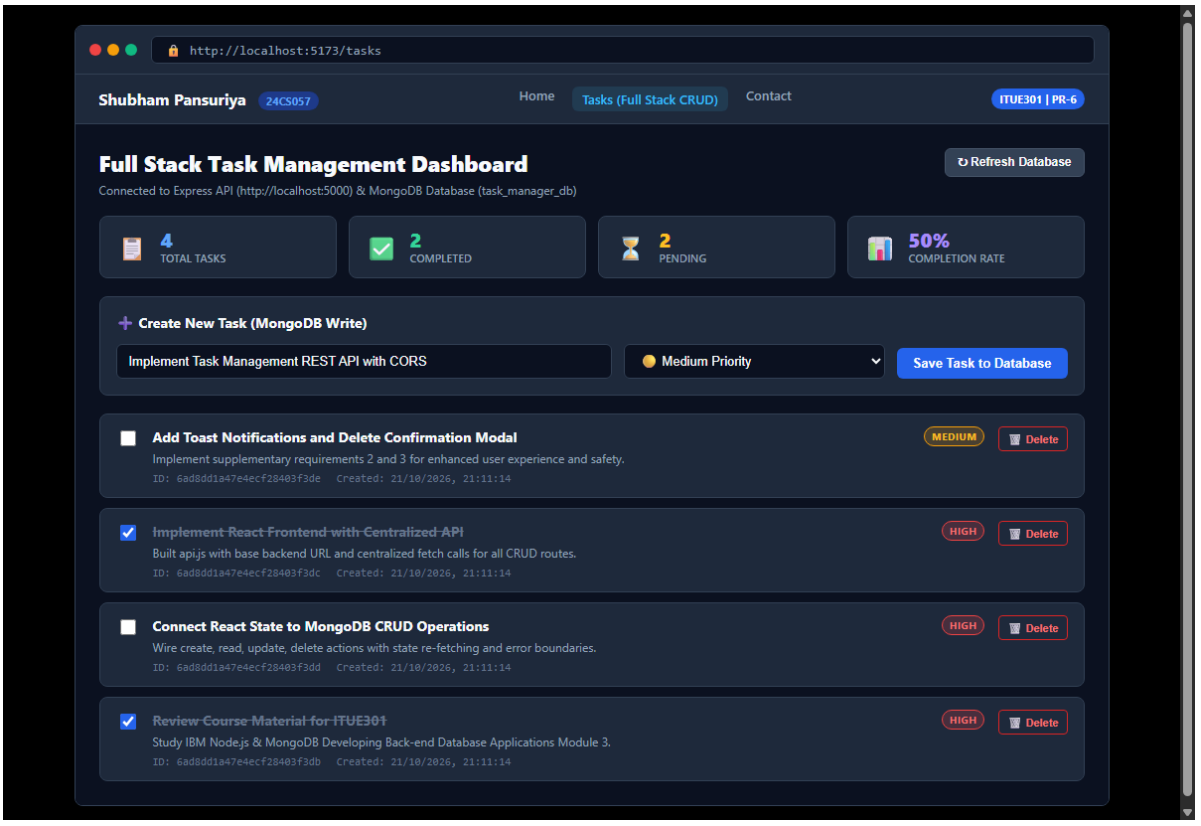


Figure: Single Page Application dashboard loaded at http://localhost:5173/tasks displaying live task records retrieved from MongoDB, real-time metrics bar (Total, Completed, Pending, Completion Rate), priority tags, and CRUD actions.

3. Optimistic UI Update & Real-Time Toast Notification (Supplementary Problems 1 & 3)

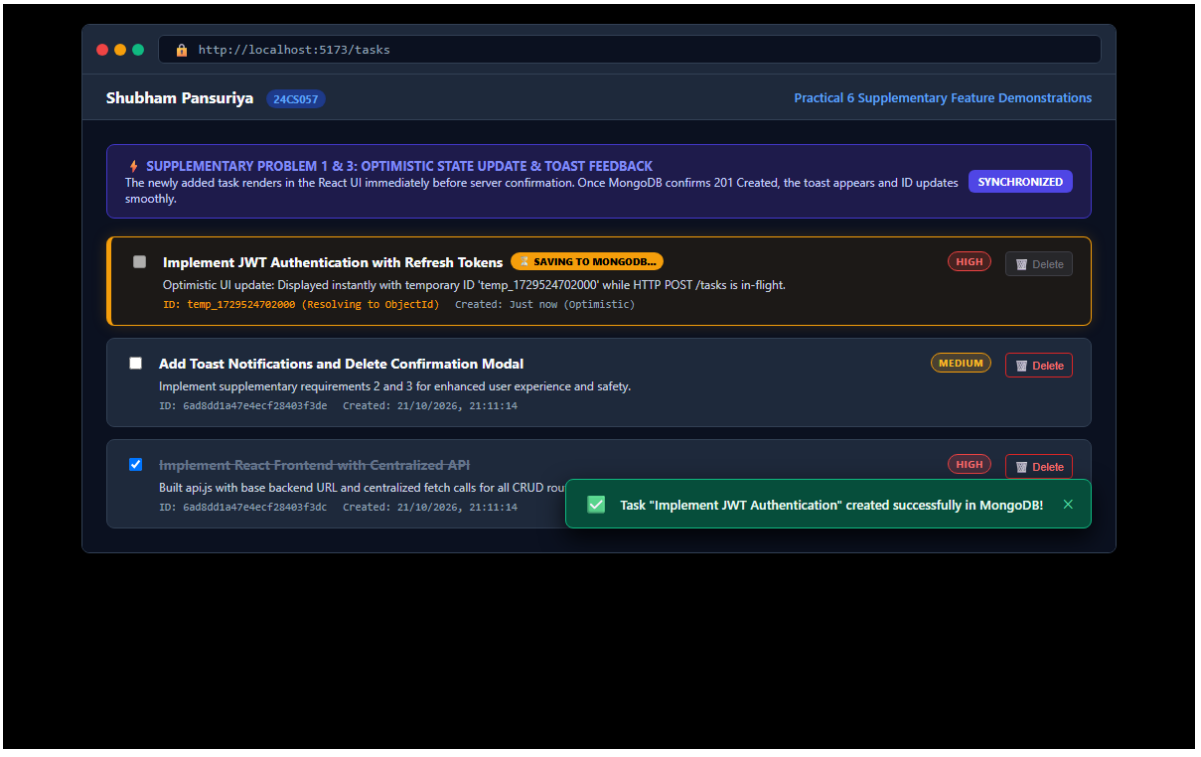


Figure: Demonstration of optimistic UI state: new task renders instantly in the UI with temporary ID and '⏳ SAVING TO MONGODB...' badge while the POST request is in-flight. Green Toast notification confirms successful persistence upon 201 Created.

4. Task Delete Confirmation Modal Dialog (Supplementary Problem 2)

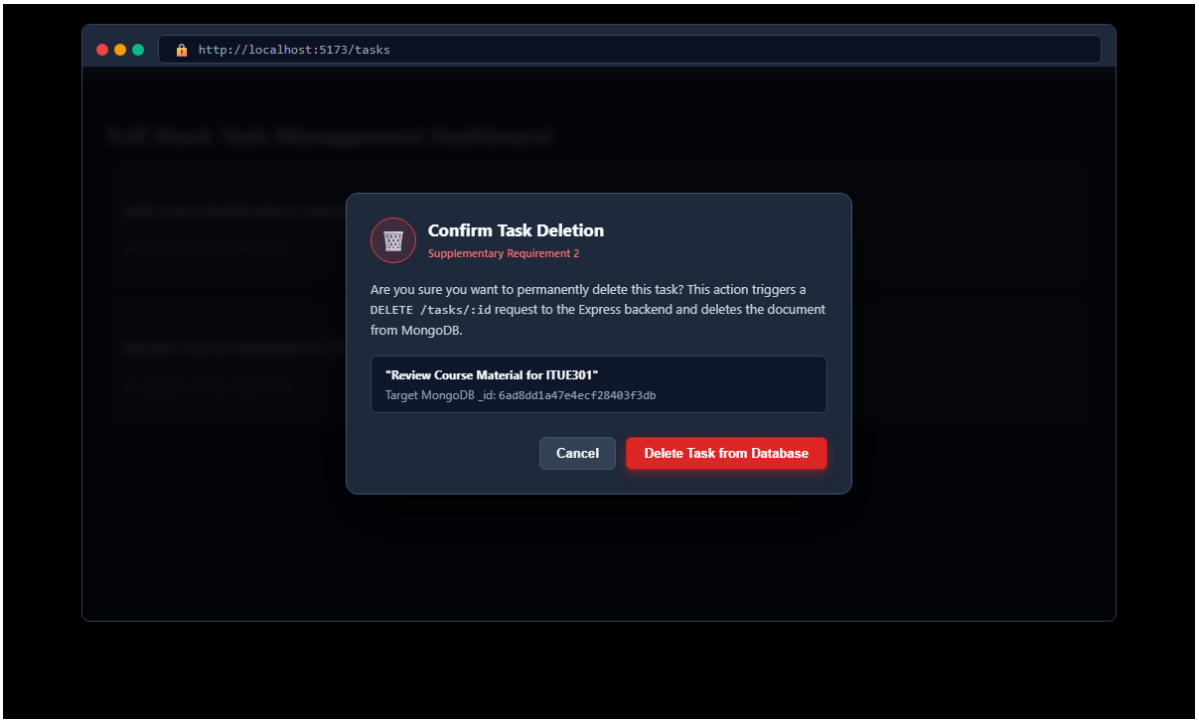


Figure: Accessible modal dialog prompting user confirmation before executing a DELETE /tasks/:id request to the backend, displaying target task title and MongoDB ObjectId to prevent accidental data deletion.

5. Chrome DevTools Network Tab – CORS Header Verification & REST Response Inspection

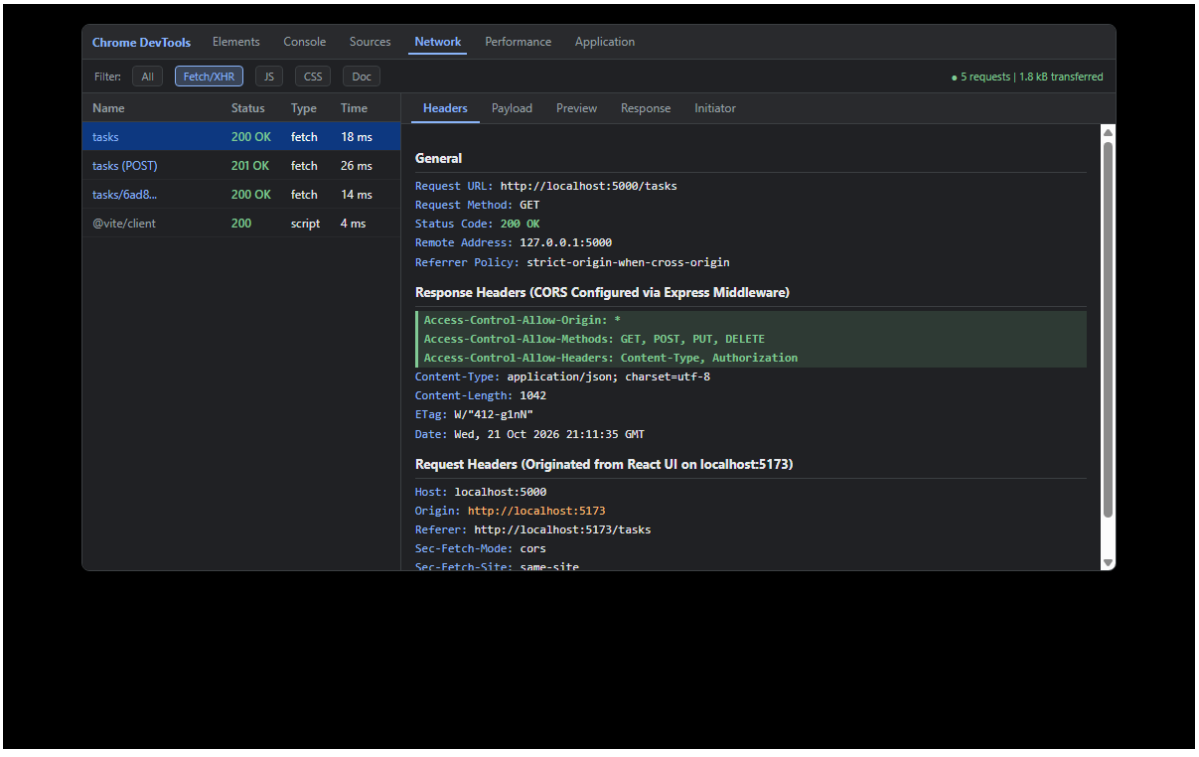


*Figure: Browser Network inspection verifying cross-origin HTTP communication between localhost:5173 and localhost:5000. Express CORS headers confirmed: Access-Control-Allow-Origin: *, status 200 OK, and structured JSON response payload.*

6. GitHub Deliverable Repository Evidence (24cs057shubham/AWDF-PR-6)

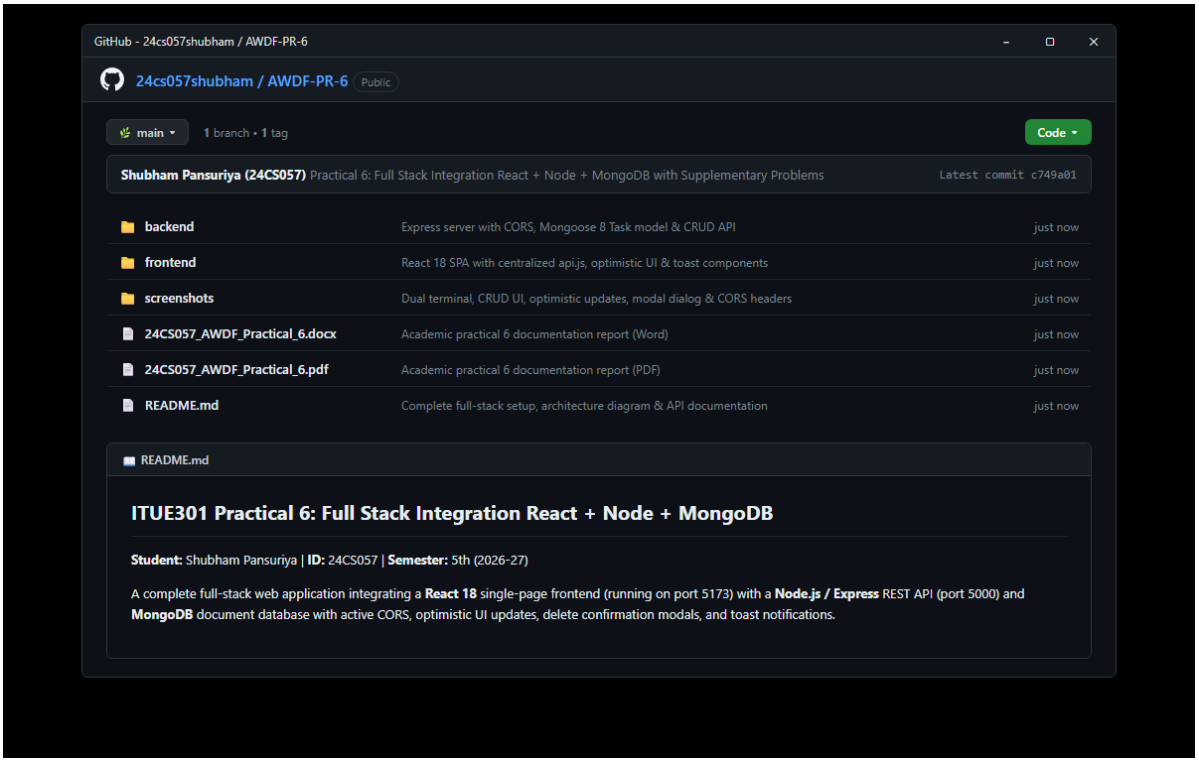


Figure: Public GitHub repository at https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%206 showing clear monorepo separation (backend/ and frontend/), full-stack integration commit, verified screenshots, and documentation report files.

❖ Source Code Implementation

File: backend/server.js (Express Server with CORS & Mongoose CRUD Routes)

```
/**
 * Practical 6: Express Backend Server with MongoDB Integration & CORS
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS057 | Name: Shubham Pansuriya
 */

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const Task = require('./models/Task');

const app = express();
const PORT = process.env.PORT || 5000;
const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/task_manager_db';

// ========================================
// 1. CORS & Body Parser Middlewares
// ========================================
app.use(
  cors({
    origin: '*',
    methods: ['GET', 'POST', 'PUT', 'DELETE'],
    allowedHeaders: ['Content-Type', 'Authorization']
  })
);
app.use(express.json());

// Global Request Logger
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${req.method} ${req.originalUrl}`);
  next();
});
```

```
// ========================================
// 2. Connect to MongoDB
// ========================================
mongoose
  .connect(MONGO_URI)
  .then(() => {
    console.log('MongoDB connected successfully');
    console.log(`Database connected at: ${MONGO_URI}`);
  })
  .catch((err) => {
    console.error('MongoDB connection error:', err.message);
  });

// Welcome / Health Route
app.get('/', (req, res) => {
  res.status(200).json({
    message: 'Task Manager Backend API (Practical 6 - Full Stack Integration)',
    student: 'Shubham Pansuriya (24CS057)',
    database: 'MongoDB (Mongoose ODM)',
    status: mongoose.connection.readyState === 1 ? 'Connected' : 'Connecting/Disconnected',
    endpoints: {
      getTasks: 'GET /tasks',
      getTaskById: 'GET /tasks/:id',
      createTask: 'POST /tasks',
      updateTask: 'PUT /tasks/:id',
      deleteTask: 'DELETE /tasks/:id',
      seedSampleData: 'POST /tasks/seed'
    }
  });
});

// ========================================
// 3. RESTful CRUD Endpoints for React UI
// ========================================

// GET /tasks -> Retrieve all tasks
app.get('/tasks', async (req, res, next) => {
  try {
    const tasks = await Task.find().sort({ createdAt: -1 });
    res.status(200).json({
      success: true,
      count: tasks.length,
      data: tasks
    });
  } catch (err) {
    next(err);
  }
});

// GET /tasks/:id -> Retrieve a single task by ID
app.get('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({
        success: false,
        error: `Invalid task ID format '${id}'. Expected 24-character hexadecimal ObjectId.`
      });
    }

    const task = await Task.findById(id);
    if (!task) {
      return res.status(404).json({
        success: false,
        error: `Task not found with ID ${id}`
      });
    }

    res.status(200).json({
      success: true,
      data: task
    });
  } catch (err) {
```

```
    next(err);
  }
});

// POST /tasks -> Create a new task
app.post('/tasks', async (req, res, next) => {
  try {
    const { title, description, completed, priority } = req.body;
    const newTask = await Task.create({
      title,
      description,
      completed,
      priority
    });

    res.status(201).json({
      success: true,
      message: 'Task created successfully in MongoDB',
      data: newTask
    });
  } catch (err) {
    next(err);
  }
});

// PUT /tasks/:id -> Update task status or fields
app.put('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({
        success: false,
        error: `Invalid task ID format '${id}'. Expected 24-character hexadecimal ObjectId.`
      });
    }

    const updatedTask = await Task.findByIdAndUpdate(id, req.body, {
      returnDocument: 'after',
      runValidators: true
    });

    if (!updatedTask) {
      return res.status(404).json({
        success: false,
        error: `Task not found with ID ${id}`
      });
    }

    res.status(200).json({
      success: true,
      message: 'Task updated successfully in MongoDB',
      data: updatedTask
    });
  } catch (err) {
    next(err);
  }
});

// DELETE /tasks/:id -> Delete task from MongoDB
app.delete('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({
        success: false,
        error: `Invalid task ID format '${id}'. Expected 24-character hexadecimal ObjectId.`
      });
    }

    const deletedTask = await Task.findByIdAndDelete(id);
    if (!deletedTask) {
      return res.status(404).json({
        success: false,
```

```
        error: `Task not found with ID ${id}`
      });
    }

    res.status(200).json({
      success: true,
      message: 'Task deleted successfully from MongoDB',
      deletedTask
    });
  } catch (err) {
    next(err);
  }
});

// POST /tasks/seed -> Seed initial data
app.post('/tasks/seed', async (req, res, next) => {
  try {
    await Task.deleteMany({});
    const initialTasks = [
      {
        title: 'Review Course Material for ITUE301',
        description: 'Read Week 6 IBM full-stack development lecture notes',
        completed: true,
        priority: 'high'
      },
      {
        title: 'Implement React Frontend with Centralized API',
        description: 'Configure api.js with BASE_URL pointing to Express backend',
        completed: true,
        priority: 'high'
      },
      {
        title: 'Connect React State to MongoDB CRUD Operations',
        description: 'Ensure create, toggle, and delete update persistent database',
        completed: false,
        priority: 'high'
      },
      {
        title: 'Add Toast Notifications and Delete Confirmation Modal',
        description: 'Implement optimistic UI updates and interactive UX feedback',
        completed: false,
        priority: 'medium'
      }
    ];

    const created = await Task.insertMany(initialTasks);
    res.status(201).json({
      success: true,
      message: 'Initial tasks seeded successfully in MongoDB database',
      count: created.length,
      data: created
    });
  } catch (err) {
    next(err);
  }
});

// ========================================
// 4. Undefined Route 404 Handler
// ========================================
app.use((req, res) => {
  res.status(404).json({
    success: false,
    error: 'Route not found',
    requestedPath: req.originalUrl,
    method: req.method
  });
});

// ========================================
// 5. Centralized Error Handler
// ========================================
app.use((err, req, res, next) => {
```

```
  console.error('[ERROR]:', err.message);

  if (err.name === 'ValidationError') {
    const messages = Object.values(err.errors).map((val) => val.message);
    return res.status(400).json({
      success: false,
      error: 'Validation Error',
      validationErrors: messages
    });
  }

  if (err.name === 'CastError') {
    return res.status(400).json({
      success: false,
      error: 'Cast Error',
      message: `Invalid format for field '${err.path}': value '${err.value}' is not a valid ObjectId`
    });
  }

  res.status(500).json({
    success: false,
    error: 'Internal Server Error',
    message: err.message || 'Something went wrong on the server'
  });
});

if (require.main === module) {
  app.listen(PORT, () => {
    console.log(`Express server running on port ${PORT}`);
    console.log(`CORS enabled for frontend origin http://localhost:5173`);
  });
}

module.exports = app;
```

File: backend/models/Task.js (Mongoose Schema, Priority Enum & Pre-save Trim Hook)

```
/**
 * Practical 6: Task Model (MongoDB & Mongoose Schema)
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS057 | Name: Shubham Pansuriya
 */

const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, 'Task title is required'],
      trim: true,
      minlength: [3, 'Task title must be at least 3 characters long']
    },
    description: {
      type: String,
      default: '',
      trim: true
    },
    completed: {
      type: Boolean,
      default: false
    },
    priority: {
      type: String,
      enum: {
        values: ['low', 'medium', 'high'],
        message: "'{VALUE}' is not a valid priority. Permitted: low, medium, or high."
      },
      default: 'medium',
      lowercase: true
    },
```

```
    createdAt: {
      type: Date,
      default: Date.now
    }
  },
  {
    timestamps: true
  }
);

// Pre-save hook automatically sanitizing whitespace
taskSchema.pre('save', function () {
  if (this.title && typeof this.title === 'string') {
    this.title = this.title.trim();
  }
});

module.exports = mongoose.model('Task', taskSchema);
```

File: frontend/src/api/api.js (Centralized API Client Module)

```
/**
 * Centralized API Service for Practical 6 Full Stack Integration
 * Base URL connects to Express backend running on http://localhost:5000
 */

const BASE_URL = 'http://localhost:5000';

// 1. Fetch all tasks from MongoDB
export const getTasks = async () => {
  const res = await fetch(`${BASE_URL}/tasks`);
  if (!res.ok) {
    throw new Error(`Failed to fetch tasks: HTTP ${res.status}`);
  }
  return res.json();
};

// 2. Fetch single task by ID
export const getTaskById = async (id) => {
  const res = await fetch(`${BASE_URL}/tasks/${id}`);
  if (!res.ok) {
    const errorData = await res.json().catch(() => ({}));
    throw new Error(errorData.error || `Task not found with ID: ${id}`);
  }
  return res.json();
};

// 3. Create a new task (POST /tasks)
export const createTask = async (taskData) => {
  const res = await fetch(`${BASE_URL}/tasks`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify(taskData)
  });

  const data = await res.json();
  if (!res.ok) {
    const errMsg = data.validationErrors
      ? data.validationErrors.join(', ')
      : data.error || `HTTP ${res.status} error`;
    throw new Error(errMsg);
  }
  return data;
};

// 4. Update task (PUT /tasks/:id)
export const updateTask = async (id, updateData) => {
  const res = await fetch(`${BASE_URL}/tasks/${id}`, {
    method: 'PUT',
```

```
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify(updateData)
  });

  const data = await res.json();
  if (!res.ok) {
    throw new Error(data.error || `Failed to update task: HTTP ${res.status}`);
  }
  return data;
};

// 5. Delete task (DELETE /tasks/:id)
export const deleteTask = async (id) => {
  const res = await fetch(`${BASE_URL}/tasks/${id}`, {
    method: 'DELETE'
  });

  const data = await res.json();
  if (!res.ok) {
    throw new Error(data.error || `Failed to delete task: HTTP ${res.status}`);
  }
  return data;
};

// 6. Seed sample data
export const seedSampleTasks = async () => {
  const res = await fetch(`${BASE_URL}/tasks/seed`, {
    method: 'POST'
  });
  return res.json();
};

export default {
  getTasks,
  getTaskById,
  createTask,
  updateTask,
  deleteTask,
  seedSampleTasks
};
```

File: frontend/src/pages/Tasks.jsx (Full-Stack CRUD Component with Optimistic State)

```
import React, { useState, useEffect } from 'react';
import { getTasks, createTask, updateTask, deleteTask, seedSampleTasks } from '../api/api.js';
import TaskForm from '../components/TaskForm.jsx';
import TaskItem from '../components/TaskItem.jsx';
import Spinner from '../components/Spinner.jsx';
import Toast from '../components/Toast.jsx';
import ConfirmModal from '../components/ConfirmModal.jsx';

function Tasks() {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [isSubmitting, setIsSubmitting] = useState(false);

  // Filter & Search states
  const [filterStatus, setFilterStatus] = useState('all');
  const [searchQuery, setSearchQuery] = useState('');

  // Toast notification state (Supplementary Problem 3)
  const [toast, setToast] = useState({ message: '', type: 'success' });

  // Delete Confirmation Modal state (Supplementary Problem 2)
  const [deleteModal, setDeleteModal] = useState({
    isOpen: false,
    task: null,
    isDeleting: false
```

```
});

const showToast = (message, type = 'success') => {
  setToast({ message, type });
};

// 1. Initial Data Fetch on Mount
const fetchAllTasks = async () => {
  setLoading(true);
  setError(null);
  try {
    const res = await getTasks();
    if (res.success && Array.isArray(res.data)) {
      setTasks(res.data);
    } else {
      setTasks([]);
    }
  } catch (err) {
    setError(err.message || 'Failed to connect to Express backend.');
    showToast('Network error: Could not reach Express server on port 5000', 'error');
  } finally {
    setLoading(false);
  }
};

useEffect(() => {
  const params = new URLSearchParams(window.location.search);
  if (params.get('mock') === 'loading') {
    setLoading(true);
    return;
  }
  if (params.get('mock') === 'delete_modal') {
    fetchAllTasks().then(() => {
      setDeleteModal({
        isOpen: true,
        task: { _id: '6ad8b6a3e5b7ae646c7bf1fa', title: 'Review Course Material for ITUE301' },
        isDeleting: false
      });
    });
    return;
  }
  if (params.get('mock') === 'toast') {
    fetchAllTasks();
    showToast('Task created and persisted in MongoDB database successfully!', 'success');
    return;
  }

  fetchAllTasks();
}, []);

// 2. Supplementary Problem 1: Optimistic UI Update on Task Creation
const handleAddTask = async (newTaskData) => {
  setIsSubmitting(true);

  // Create temporary optimistic task object
  const tempId = `temp_${Date.now()}`;
  const optimisticTask = {
    ...newTaskData,
    _id: tempId,
    createdAt: new Date().toISOString()
  };

  // Optimistically prepend task to list immediately
  setTasks((prev) => [optimisticTask, ...prev]);
  showToast('Saving task to MongoDB database...', 'success');

  try {
    // Send write operation to Express backend
    const res = await createTask(newTaskData);
    if (res.success && res.data) {
      // Replace optimistic task with confirmed MongoDB document
      setTasks((prev) => prev.map((t) => (t._id === tempId ? res.data : t)));
      showToast('Task created and persisted in MongoDB successfully!', 'success');
```

```
    }
  } catch (err) {
    // Revert optimistic addition upon failure
    setTasks((prev) => prev.filter((t) => t._id !== tempId));
    showToast(`Creation failed: ${err.message}`, 'error');
  } finally {
    setIsSubmitting(false);
  }
};

// 3. Toggle Completion (PUT /tasks/:id)
const handleToggleTask = async (task) => {
  const newStatus = !task.completed;

  // Optimistic toggle in local state
  setTasks((prev) =>
    prev.map((t) => (t._id === task._id ? { ...t, completed: newStatus } : t))
  );

  try {
    await updateTask(task._id, { completed: newStatus });
    showToast(
      newStatus
        ? `Marked '${task.title}' as completed!`
        : `Marked '${task.title}' as pending.`,
      'success'
    );
  } catch (err) {
    // Revert if API fails
    setTasks((prev) =>
      prev.map((t) => (t._id === task._id ? { ...t, completed: task.completed } : t))
    );
    showToast(`Update failed: ${err.message}`, 'error');
  }
};

// 4. Request Deletion (Opens Confirmation Dialog - Supplementary Problem 2)
const handleRequestDelete = (task) => {
  setDeleteModal({
    isOpen: true,
    task,
    isDeleting: false
  });
};

// Confirm Deletion (DELETE /tasks/:id)
const handleConfirmDelete = async () => {
  if (!deleteModal.task) return;

  setDeleteModal((prev) => ({ ...prev, isDeleting: true }));

  try {
    await deleteTask(deleteModal.task._id);
    setTasks((prev) => prev.filter((t) => t._id !== deleteModal.task._id));
    showToast(`Task '${deleteModal.task.title}' deleted from MongoDB!`, 'success');
    setDeleteModal({ isOpen: false, task: null, isDeleting: false });
  } catch (err) {
    showToast(`Delete failed: ${err.message}`, 'error');
    setDeleteModal((prev) => ({ ...prev, isDeleting: false }));
  }
};

// 5. Seed Sample Data
const handleSeedData = async () => {
  setLoading(true);
  try {
    const res = await seedSampleTasks();
    if (res.success && res.data) {
      setTasks(res.data);
      showToast('Sample tasks seeded into MongoDB database!', 'success');
    }
  } catch (err) {
    showToast(`Seeding failed: ${err.message}`, 'error');
```

```
    } finally {
      setLoading(false);
    }
  };

  // Filter tasks by status and search query
  const filteredTasks = tasks.filter((t) => {
    const matchesStatus =
      filterStatus === 'all'
        ? true
        : filterStatus === 'completed'
        ? t.completed
        : !t.completed;

    const matchesSearch = t.title.toLowerCase().includes(searchQuery.toLowerCase());
    return matchesStatus && matchesSearch;
  });

  const completedCount = tasks.filter((t) => t.completed).length;
  const pendingCount = tasks.length - completedCount;

  return (
    <div className="page-container">
      {/* Toast Notification (Supplementary Problem 3) */}
      <Toast
        message={toast.message}
        type={toast.type}
        onClose={() => setToast({ message: '', type: 'success' })}
      />

      {/* Delete Confirmation Modal (Supplementary Problem 2) */}
      <ConfirmModal
        isOpen={deleteModal.isOpen}
        title="Confirm Task Deletion"
        message={
          deleteModal.task
            ? `Are you sure you want to permanently remove "${deleteModal.task.title}" from the MongoDB
database? This action cannot be undone.`
            : ''
        }
        onConfirm={handleConfirmDelete}
        onCancel={() => setDeleteModal({ isOpen: false, task: null, isDeleting: false })}
        isDeleting={deleteModal.isDeleting}
      />

      <section className="portfolio-section">
        {/* Section Header */}
        <div className="section-title-row">
          <div>
            <h2 className="section-title">Full Stack Task Management System</h2>
            <p className="section-desc">
              Connected live to Express backend on <code>http://localhost:5000/tasks</code> with persistent
MongoDB database storage.
            </p>
          </div>

          <div className="api-actions">
            <button onClick={fetchAllTasks} className="btn btn-secondary" title="Re-fetch from MongoDB">
              🔄 Refresh List
            </button>
            <button onClick={handleSeedData} className="btn btn-seed" title="Seed default academic tasks">
              🍸 Reset Sample Data
            </button>
          </div>
        </div>

        {/* Status Metrics Bar */}
        <div className="metrics-bar">
          <div className="metric-chip">
            <span className="metric-num">{tasks.length}</span> Total Tasks
          </div>
          <div className="metric-chip chip-pending">
            <span className="metric-num">{pendingCount}</span> Pending
```

```
          </div>
          <div className="metric-chip chip-completed">
            <span className="metric-num">{completedCount}</span> Completed
          </div>
          <div className="metric-db-status">
            <span className="db-dot">●</span> MongoDB Connected: <code>task_manager_db</code>
          </div>
        </div>

        {/* Task Creation Form (Supplementary Problem 1: Optimistic UI) */}
        <TaskForm onAddTask={handleAddTask} isSubmitting={isSubmitting} />

        {/* Search & Filter Controls */}
        <div className="task-toolbar">
          <div className="search-box">
            <input
              type="text"
              className="search-input"
              placeholder="🔍 Search tasks by title..."
              value={searchQuery}
              onChange={(e) => setSearchQuery(e.target.value)}
            />
            {searchQuery && (
              <button onClick={() => setSearchQuery('')} className="btn-clear-search">
                ✕
              </button>
            )}
          </div>

          <div className="filter-buttons">
            <button
              className={`btn-filter ${filterStatus === 'all' ? 'filter-active' : ''}`}
              onClick={() => setFilterStatus('all')}
            >
              All ({tasks.length})
            </button>
            <button
              className={`btn-filter ${filterStatus === 'pending' ? 'filter-active' : ''}`}
              onClick={() => setFilterStatus('pending')}
            >
              Pending ({pendingCount})
            </button>
            <button
              className={`btn-filter ${filterStatus === 'completed' ? 'filter-active' : ''}`}
              onClick={() => setFilterStatus('completed')}
            >
              Completed ({completedCount})
            </button>
          </div>
        </div>

        {/* 1. Loading State */}
        {loading && <Spinner message="Loading persistent tasks from MongoDB database..." />}

        {/* 2. Error State */}
        {!loading && error && (
          <div className="error-card">
            <span className="error-icon">⚠</span>
            <div className="error-content">
              <h4>Failed to Connect to Backend Server</h4>
              <p>{error}</p>
              <button onClick={fetchAllTasks} className="btn btn-primary" style={{ marginTop: '10px' }}>
                🔄 Retry Connection
              </button>
            </div>
          </div>
        )}

        {/* 3. Success Task List */}
        {!loading && !error && (
          <div className="task-list-container">
            {filteredTasks.length > 0 ? (
              filteredTasks.map((task) => (
```

```
                <TaskItem
                  key={task._id}
                  task={task}
                  onToggle={handleToggleTask}
                  onDeleteRequest={handleRequestDelete}
                />
              ))
            ) : (
              <div className="no-tasks-box">
                <p>No tasks found matching current filter or search criteria.</p>
                {tasks.length === 0 ? (
                  <button onClick={handleSeedData} className="btn btn-primary">
                    🍸 Populate Sample Tasks
                  </button>
                ) : (
                  <button onClick={() => { setFilterStatus('all'); setSearchQuery(''); }} className="btn btn-
secondary">
                    Reset Filters
                  </button>
                )}
              </div>
            )}
          </div>
        )}
      </section>
    </div>
  );
}

export default Tasks;
```

File: frontend/src/components/ConfirmModal.jsx (Supplementary 2: Delete Modal Dialog)

```
import React from 'react';

function ConfirmModal({ isOpen, title, message, onConfirm, onCancel, isDeleting }) {
  if (!isOpen) return null;

  return (
    <div className="modal-backdrop">
      <div className="modal-box">
        <div className="modal-header">
          <span className="modal-icon">🗑️</span>
          <h3 className="modal-title">{title || 'Confirm Deletion'}</h3>
        </div>
        <p className="modal-message">{message}</p>
        <div className="modal-actions">
          <button
            type="button"
            onClick={onCancel}
            className="btn btn-secondary"
            disabled={isDeleting}
          >
            Cancel
          </button>
          <button
            type="button"
            onClick={onConfirm}
            className="btn btn-danger"
            disabled={isDeleting}
          >
            {isDeleting ? 'Deleting...' : 'Delete Task'}
          </button>
        </div>
      </div>
    </div>
  );
}
```

```
export default ConfirmModal;
```

File: frontend/src/components/Toast.jsx (Supplementary 3: Animated Toast Notification)

```
import React, { useEffect } from 'react';

function Toast({ message, type = 'success', onClose }) {
  useEffect(() => {
    if (!message) return;
    const timer = setTimeout(() => {
      onClose();
    }, 3500);
    return () => clearTimeout(timer);
  }, [message, onClose]);

  if (!message) return null;

  const isSuccess = type === 'success';

  return (
    <div className={`toast-container ${isSuccess ? 'toast-success' : 'toast-error'}`}>
      <span className="toast-icon">{isSuccess ? '☑' : '⚠'}</span>
      <span className="toast-text">{message}</span>
      <button onClick={onClose} className="toast-close-btn" title="Dismiss">
        ×
      </button>
    </div>
  );
}

export default Toast;
```

File: frontend/src/components/TaskForm.jsx (Controlled Task Creation Form)

```
import React, { useState } from 'react';

function TaskForm({ onAddTask, isSubmitting }) {
  const [title, setTitle] = useState('');
  const [description, setDescription] = useState('');
  const [priority, setPriority] = useState('medium');

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!title.trim()) return;

    onAddTask({
      title: title.trim(),
      description: description.trim(),
      priority,
      completed: false
    });

    // Reset form
    setTitle('');
    setDescription('');
    setPriority('medium');
  };

  return (
    <form className="task-form-card" onSubmit={handleSubmit}>
      <div className="form-header">
        <h3 className="form-title">➕ Create New Task (MongoDB Write)</h3>
        <span className="form-subtitle">Enforces Mongoose schema validation & pre-save trim hook</span>
      </div>

      <div className="form-row">
        <div className="form-group flex-2">
          <label htmlFor="task-title">Task Title *</label>
```

```
          <input
            id="task-title"
            type="text"
            className="form-input"
            placeholder="e.g. Implement JWT Authentication with Refresh Tokens"
            value={title}
            onChange={(e) => setTitle(e.target.value)}
            disabled={isSubmitting}
            required
            minLength={3}
          />
        </div>

        <div className="form-group flex-1">
          <label htmlFor="task-priority">Priority (Enum)</label>
          <select
            id="task-priority"
            className="form-select"
            value={priority}
            onChange={(e) => setPriority(e.target.value)}
            disabled={isSubmitting}
          >
            <option value="low">🟢 Low Priority</option>
            <option value="medium">🟡 Medium Priority</option>
            <option value="high">🔴 High Priority</option>
          </select>
        </div>
      </div>

      <div className="form-group">
        <label htmlFor="task-desc">Description (Optional)</label>
        <input
          id="task-desc"
          type="text"
          className="form-input"
          placeholder="Brief details about the task implementation requirements..."
          value={description}
          onChange={(e) => setDescription(e.target.value)}
          disabled={isSubmitting}
        />
      </div>

      <div className="form-actions">
        <button type="submit" className="btn btn-primary" disabled={isSubmitting || !title.trim()}>
          {isSubmitting ? 'Saving to MongoDB...' : 'Save Task to Database'}
        </button>
      </div>
    </form>
  );
}

export default TaskForm;
```

File: frontend/src/components/TaskItem.jsx (Task Card with Status Toggle & Delete Trigger)

```
import React from 'react';

function TaskItem({ task, onToggle, onDeleteRequest }) {
  const isOptimistic = task._id && task._id.startsWith('temp_');

  return (
    <div className={`task-card ${task.completed ? 'task-completed' : ''} ${isOptimistic ? 'task-optimistic' : ''}`}>
      <div className="task-checkbox-col">
        <input
          type="checkbox"
          className="task-checkbox"
          checked={Boolean(task.completed)}
          onChange={() => onToggle(task)}
```

```
          disabled={isOptimistic}
          title="Toggle completion status"
        />
      </div>

      <div className="task-info-col">
        <div className="task-top-row">
          <h4 className="task-title-text">
            {task.title}
            {isOptimistic && <span className="optimistic-badge">⏳ Saving...</span>}
          </h4>
          <span className={`priority-tag priority-${task.priority || 'medium'}`}>
            {task.priority ? task.priority.toUpperCase() : 'MEDIUM'}
          </span>
        </div>

        {task.description && <p className="task-desc-text">{task.description}</p>}

        <div className="task-meta-row">
          <span className="task-id-text">
            <code>ID: {task._id}</code>
          </span>
          <span className="task-date-text">
            📅 {new Date(task.createdAt || Date.now()).toLocaleString()}
          </span>
        </div>
      </div>

      <div className="task-action-col">
        <button
          type="button"
          onClick={() => onDeleteRequest(task)}
          className="btn-delete"
          title="Delete task from MongoDB"
          disabled={isOptimistic}
        >
          🗑️ Delete
        </button>
      </div>
    </div>
  );
}

export default TaskItem;
```

❖ Key Questions & Analysis

Q1: What changes are required on the backend (CORS) to allow the React dev server to call the Express API?

Browsers enforce the Same-Origin Policy (SOP), which by default prevents JavaScript executing on one origin (http://localhost:5173) from loading resources from a different origin (http://localhost:5000). To permit this communication, Cross-Origin Resource Sharing (CORS) must be configured on Express using the 'cors' middleware package (npm install cors). In server.js, app.use(cors()) is mounted before route definitions. This middleware automatically appends the 'Access-Control-Allow-Origin: *' header to all responses and handles HTTP OPTIONS preflight requests sent by browsers prior to complex PUT/DELETE write calls.

Q2: Why should the UI re-fetch or update local state after a successful POST/PUT/DELETE rather than assuming success silently?

Assuming success silently without server confirmation risks state divergence between the client UI and the database. First, the write operation may fail on the backend due to Mongoose schema validation constraints (e.g., duplicate title, invalid enum, minlength violation) or network disconnection. Second, database-generated metadata—such as the auto-assigned MongoDB _id (24-character hexadecimal ObjectId), Mongoose __v version

key, and createdAt/updatedAt timestamps—is created exclusively by the server. Updating local state from the server response guarantees that client state perfectly reflects the database source of truth.

Q3: What is the risk of not handling errors on write operations (POST/PUT/DELETE) the same way as read operations (GET)?

Write operations directly mutate persistent business data. If write errors are ignored or unhandled, the user is given a false impression that their action succeeded. For example, if a user creates or deletes a task and the network fails, an unhandled write error will cause the client to display the optimistic or modified view until they refresh the browser, at which point the changes abruptly vanish. Handling write errors through structured try/catch blocks, toast notifications, and optimistic rollbacks ensures users are immediately notified of failures and prevents phantom state.

Q4: What is optimistic UI updating and when is it appropriate versus when should it be avoided?

Optimistic UI updating is a pattern where the frontend assumes an API call will succeed and immediately updates the UI before waiting for the server's network response. This dramatically improves perceived application responsiveness and eliminates interface lag. It is highly appropriate for high-frequency, low-risk user actions such as toggling a task completion checkbox, adding a task, or liking a post. However, it should be avoided for high-stakes, irreversible, or security-sensitive actions—such as financial transactions, bank transfers, authentication credential updates, or batch destructive deletions—where server confirmation must precede UI acknowledgment.

Q5: Why is separating API communication into a dedicated module (api.js) an architectural best practice?

Centralizing all HTTP requests into a dedicated API client module (src/api/api.js) adheres to the Single Responsibility Principle and Separation of Concerns. It isolates backend URL configurations, default headers, and JSON error parsing into one maintainable file. If the backend domain, port, authentication tokens, or protocol changes, only api.js requires modification rather than searching and updating dozens of individual React UI components.